



BM CỤNG NGHỆ PHẦN MỀM — KHOA CNTT
SE.NUCE.EDU.VN

CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN NÂNG CAO

Chương 3. Tìm kiếm

Chương 3. Tìm kiếm

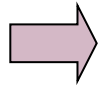
3.1. Giới thiệu

3.2. Tìm kiếm tuần tự

3.3. Tìm kiếm nhị phân

3.4. Tìm kiếm theo bảng băm

Chương 3. Tìm kiếm



3.1. Giới thiệu

3.2. Tìm kiếm tuần tự

3.3. Tìm kiếm nhị phân

3.4. Tìm kiếm theo bảng băm

3.1. Giới thiệu

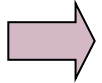
- Thao tác tìm kiếm rất phổ biến trong cuộc sống hàng ngày.
 - Tìm kiếm hồ sơ, tập tin.
 - Tìm kiếm tên người trong danh sách.
 - ...

3.1. Giới thiệu

- Có nhiều loại thuật toán tìm kiếm
 - Tìm kiếm tuần tự
 - Tìm kiếm nhị phân
 - ...
- Mục tiêu
 - Tìm hiểu về các thuật toán tìm kiếm
 - Phân tích thuật toán để lựa chọn thuật toán phù hợp khi áp dụng vào thực tế

Chương 3. Tìm kiếm

3.1. Giới thiệu



3.2. Tìm kiếm tuần tự

3.3. Tìm kiếm nhị phân

3.4. Tìm kiếm theo bảng băm

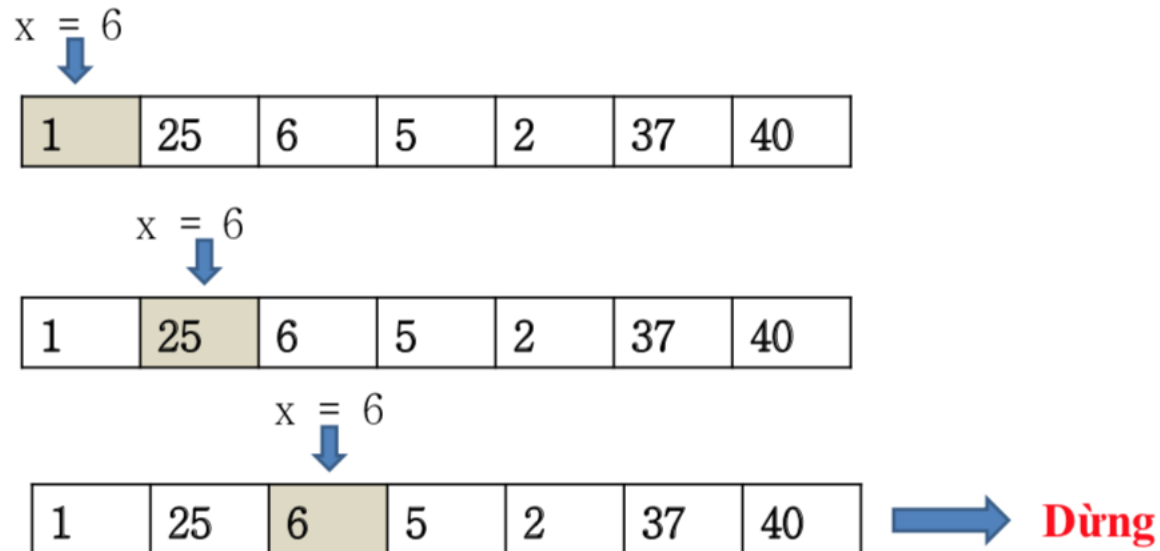
3.2. Tìm kiếm tuần tự

- Đầu vào
 - Dãy A gồm n phần tử
 - Giá trị x cần tìm
- Đầu ra
 - Nếu x xuất hiện trong A: trả về vị trí xuất hiện đầu tiên của x
 - Nếu không: trả về n hoặc -1
- Thuật toán
 - Vét cạn
 - Dùng lính canh

3.2. Tìm kiếm tuần tự - Vết cạn

- Thuật toán

- Lần lượt so sánh x với các phần tử của mảng A cho đến khi gặp được phần tử cần tìm, hoặc hết mảng.
- Ví dụ: $A = \{1, 25, 6, 5, 2, 37, 40\}$, $x = 6$



3.2. Tìm kiếm tuần tự - Vết cạn

- Thuật toán: Linear Exhaustive
 - Bước 1: Khởi tạo biến chỉ số $i = 0$
 - Bước 2: Kiểm tra xem có thực hiện hết mảng hay chưa: *So sánh i với n*
 - Nếu chưa hết mảng ($i < n$), sang bước 3
 - Nếu đã hết mảng ($i \geq n$), thông báo không tìm thấy giá trị x cần tìm
 - Bước 3: So sánh giá trị $a[i]$ với x cần tìm
 - Nếu $a[i]$ bằng x : Kết thúc chương trình và thông báo đã tìm thấy x .
 - Nếu $a[i]$ khác x , tăng i thêm 1 và quay lại bước 2.

3.2. Tìm kiếm tuần tự - Vết cạn

- Nhận xét: Phép so sánh là phép toán sơ cấp được dùng trong thuật toán. Suy ra, số lượng các phép so sánh sẽ là thước đo độ phức tạp của thuật toán.
- Mỗi vòng lặp có 2 điều kiện cần kiểm tra:
 - Kiểm tra cuối mảng (bước 2)
 - Kiểm tra phần tử hiện tại có bằng x? (bước 3)
- Trường hợp x nằm ở 2 biên của mảng A: rất hiếm khi xuất hiện.
- Ước lượng số vòng lặp trung bình sẽ hữu ích hơn.
- Số phép so sánh trung bình:
$$2(1+2+ \dots + n)/n = n+1$$

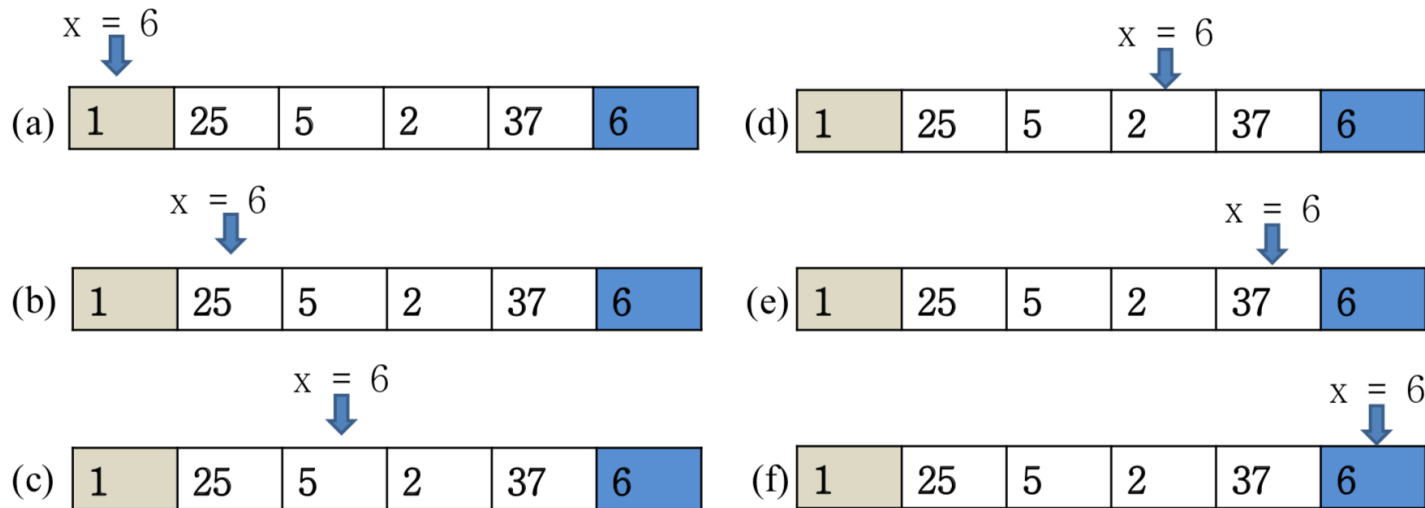
=> Số phép so sánh tăng/giảm tuyến tính theo số phần tử

3.2. Tìm kiếm tuần tự - Vết cặn

- Vậy độ phức tạp của thuật toán là:
 - Tốt nhất $O(1)$
 - Trung bình: $O(n)$.
 - Xấu nhất: $O(n)$.

3.2. Tìm kiếm tuần tự - Lính canh

- Trong thuật toán vét cạn, có 2 điều kiện được kiểm tra.
- Có thể bỏ việc kiểm tra điều kiện cuối mảng bằng cách dùng “lính canh”.
- Lính canh là phần tử có giá trị bằng với phần tử cần tìm và đặt ở cuối mảng.
- Ví dụ: $A = \{1, 25, 5, 2, 37\}$, $x = 6$



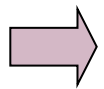
3.2. Tìm kiếm tuần tự - Lính canh

- Thuật toán: Linear Sentinel
 - Bước 1. Khởi tạo biến chỉ số: $i = 0$
 - Bước 2. So sánh giá trị $a[i]$ với giá trị x cần tìm
 - Nếu $a[i]$ bằng x :
 - Nếu $i < n$: Kết thúc chương trình và thông báo đã tìm thấy x .
 - Nếu $i \geq n$: Thông báo không tìm thấy x trong mảng.
 - Nếu $a[i]$ khác x , tăng i thêm 1 và quay lại bước 2.
- Thực nghiệm cho thấy trong trường hợp n lớn, thời gian tìm kiếm giảm khi dùng phương pháp lính canh.
 - Với $n = 15000$: nhanh hơn khoảng 20% (0,22s so với 0,28s)

Chương 3. Tìm kiếm

3.1. Giới thiệu

3.2. Tìm kiếm tuần tự



3.3. Tìm kiếm nhị phân

3.4. Tìm kiếm theo bảng băm

3.3. Tìm kiếm nhị phân

- Với dãy A được sắp xếp thứ tự (ví dụ: tăng dần), độ phức tạp của thuật toán tìm kiếm tuần tự không đổi.
- Tận dụng thông tin của mảng đã được sắp xếp để giới hạn vị trí của giá trị cần tìm trong mảng. -> Thuật toán tìm kiếm nhị phân.

3.3. Tìm kiếm nhị phân

- Đầu vào:
 - Dãy A, n phần tử đã được sắp xếp
 - Giá trị x cần tìm
- Đầu ra:
 - Nếu x xuất hiện trong A: trả về một vị trí xuất hiện của x
 - Nếu không: trả về n hoặc -1

3.3. Tìm kiếm nhị phân

- Ý tưởng:
 - So sánh x với phần tử chính giữa mảng A .
 - Nếu x là phần tử giữa thì dừng.
 - Nếu không: xác định xem x có thể thuộc nửa trái hay nửa phải của A .
 - Lặp lại 2 bước trên với nửa đã được xác định.

3.3. Tìm kiếm nhị phân

- Thuật toán: BinarySearch(A[], n, x)
 - Bước 1. Khởi gán $\text{left} = 0$ và $\text{right} = n - 1$.
 - Bước 2. Trong khi $\text{left} \leq \text{right}$, thực hiện:
 - 2.1. Đặt $\text{mid} = (\text{left} + \text{right})/2$
 - 2.2. So sánh giá trị x và $a[\text{mid}]$:
 - Nếu $x < a[\text{mid}]$, gán $\text{right} = \text{mid} - 1$.
 - Nếu $x > a[\text{mid}]$, gán $\text{left} = \text{mid} + 1$.
 - Nếu $x = a[\text{mid}]$, thông báo đã tìm thấy x và kết thúc.
 - Kết quả trả về không tìm thấy x nếu $\text{left} > \text{right}$.

3.3. Tìm kiếm nhị phân

- Cài đặt đệ quy: `BinarySearch(A[], left, right, x)`
 - Bước 1. Nếu $\text{left} > \text{right}$: thông báo không tìm thấy x và thoát khỏi hàm.
 - Bước 2.
 - 2.1. Đặt $\text{mid} = (\text{left} + \text{right})/2$
 - 2.2. So sánh giá trị x và $a[\text{mid}]$:
 - Nếu $x < a[\text{mid}]$, Gọi `BinarySearch(A, left, mid - 1, x)`
 - Nếu $x > a[\text{mid}]$, Gọi `BinarySearch(A, mid + 1, right, x)`
 - Nếu $x = a[\text{mid}]$, thông báo đã tìm thấy x và kết thúc (trả lại giá trị mid)

3.3. Tìm kiếm nhị phân

- Minh họa:
 - $A[] = \{1, 2, 6, 26, 28, 37, 40\}$, $x = 2$

index	0	1	2	3	4	5	6
$A[i]$	1	2	6	26	28	37	40
Vòng 1	left			mid			right
Vòng 2	left	mid	right				



$x = a[1] \rightarrow \text{return } 1$

3.3. Tìm kiếm nhị phân

- Minh họa:

- $A[] = \{1, 2, 6, 26, 28, 37, 40\}$, $x = 40$

index	0	1	2	3	4	5	6
A[i]	1	2	6	26	28	37	40
Vòng 1	left			mid			right
Vòng 2					left	mid	right
Vòng 3							left mid right



$x = a[6] \rightarrow \text{return } 6$

3.3. Tìm kiếm nhị phân

- Minh họa:

- $A[] = \{1, 2, 6, 26, 28, 37, 40\}$, $x = -7$

index	0	1	2	3	4	5	6
A[i]	1	2	6	26	28	37	40
Vòng 1	left			mid			right
Vòng 2	left	mid	right				
Vòng 3	left mid right						
Vòng 4							

$\text{right} = -1, \text{left} = 0$

$\Rightarrow \text{right} < \text{left} \Rightarrow$ thoát khỏi while,

return -1

3.3. Tìm kiếm nhị phân

- Độ phức tạp của tìm kiếm nhị phân
 - Trường hợp tốt nhất: $O(1)$
 - Trường hợp trung bình: $O(\log_2 n)$
 - Trường hợp xấu nhất: $O(\log_2 n)$

3.3. Tìm kiếm nhị phân

- So sánh trường hợp xấu nhất của 2 thuật toán:

Kích thước mảng	T/h xấu nhất	
	Tuần tự	Nhị phân
100.000	100.000	16
200.000	200.000	17
400.000	400.000	18
800.000	800.000	19
1.600.000	1.600.000	20

3.3. Tìm kiếm nhị phân

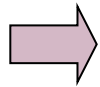
- Tổng kết
 - Có nhiều thuật toán tìm kiếm, ước lượng số phép so sánh của mỗi thuật toán cho biết hiệu suất của thuật toán.
 - Thuật toán tuần tự tìm kiếm cho đến khi tìm thấy giá trị cần tìm hoặc hết mảng
 - Hiệu suất của tìm kiếm tuần tự trong trường hợp xấu nhất là 1 hàm tuyến tính theo số phần tử mảng.
 - Nếu mảng đã được sắp xếp thì nên dùng tìm kiếm nhị phân.
 - Tìm kiếm nhị phân dùng kết quả của phép so sánh để thu hẹp vùng tìm kiếm kế tiếp.
 - Hiệu suất của tìm kiếm nhị phân là một hàm logarit theo số phần tử mảng.

Chương 3. Tìm kiếm

3.1. Giới thiệu

3.2. Tìm kiếm tuần tự

3.3. Tìm kiếm nhị phân



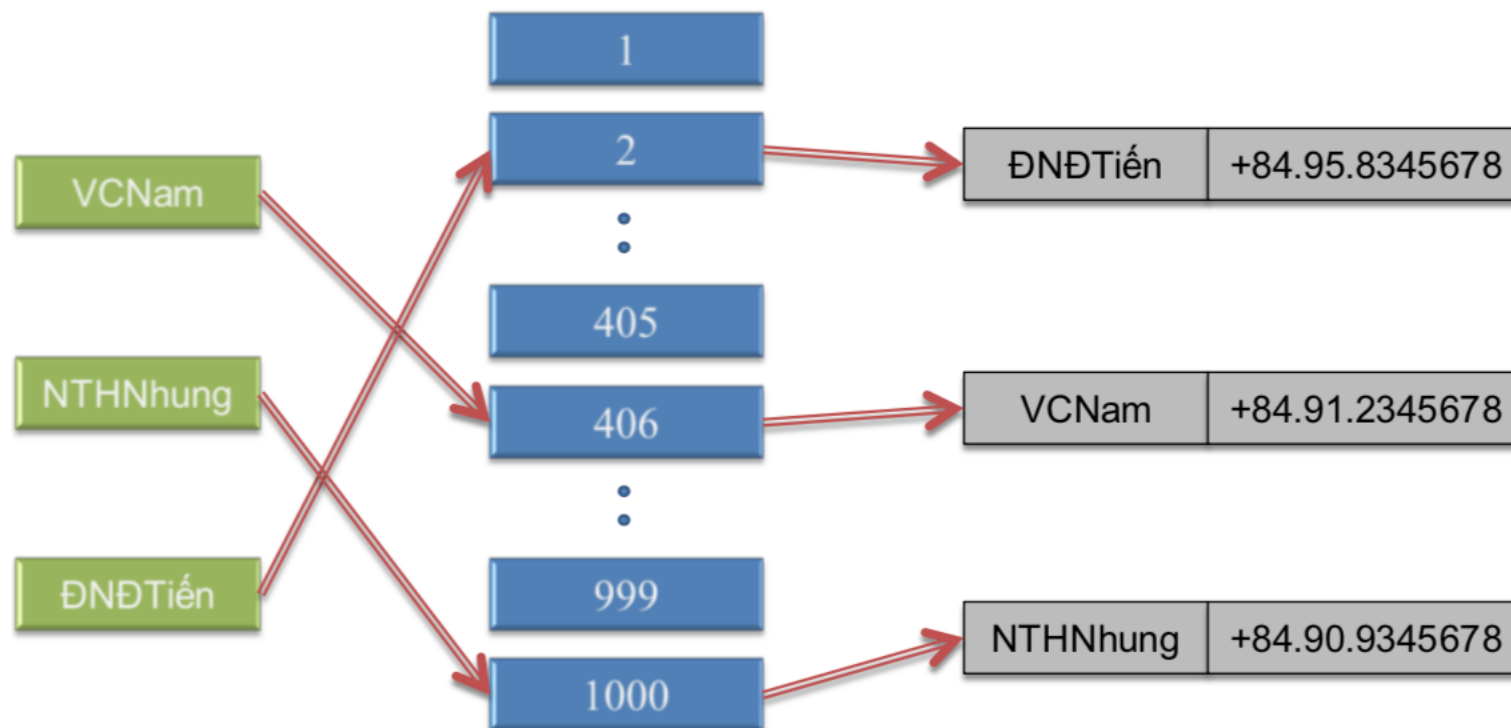
3.4. Tìm kiếm theo bảng băm

3.4. Tìm kiếm theo bảng băm

- Khái quát về hash
 - Vấn đề: Cho trước 1 tập S gồm các phần tử được đặc trưng bởi giá trị khóa. Trên giá trị các khóa này có quan hệ thứ tự. Tổ chức S như thế nào để tìm kiếm 1 phần tử có khóa k cho trước có độ phức tạp ít nhất trong giới hạn bộ nhớ cho phép?
 - Ý tưởng: Biến đổi khóa k thành một số (bằng hàm hash) và sử dụng số này như là địa chỉ để tìm kiếm trên bảng dữ liệu.

3.4. Tìm kiếm theo bảng băm

- Ví dụ



3.4. Tìm kiếm theo bảng băm

- Chi phí tìm kiếm trung bình: $O(1)$
- Chi phí tìm kiếm trong trường hợp xấu nhất: $O(n)$ (rất ít gặp).

3.4. Tìm kiếm theo bảng băm

- **Hàm băm**

- Định nghĩa: là hàm biến đổi khóa k của phần tử thành địa chỉ trong bảng băm.
- Ví dụ: $H(k) = k \bmod M$.

- **Tổng quát về phép biến đổi khóa:** Là 1 ánh xạ thích hợp từ tập các khóa U vào tập các địa chỉ A .

$$H: U \rightarrow A$$

$$k \rightarrow a = h(k)$$

3.4. Tìm kiếm theo bảng băm

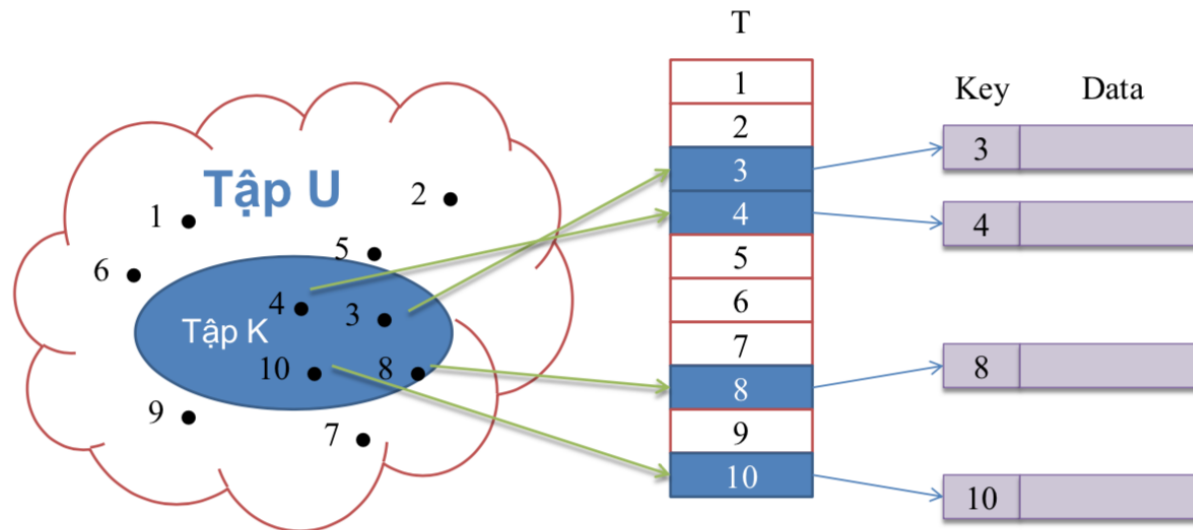
- **Khó khăn của hàm băm:**

- Tập các giá trị khóa (U) có thể lớn hơn rất nhiều so với số khóa thực tế (K) rất nhiều.
- Ví dụ: Quản lý danh sách 1000 sinh viên, mã sinh viên gồm 7 chữ số.
- Có $U = 10^7$ khóa so với $K = 1000$.

3.4. Tìm kiếm theo bảng băm

- **Khó khăn của hàm băm:**

- Tập các giá trị khóa (U) có thể lớn hơn rất nhiều so với số khóa thực tế (K) rất nhiều.
- Ví dụ: Quản lý danh sách 1000 sinh viên, mã sinh viên gồm 7 chữ số.
- Có $U = 10^7$ khóa so với $K = 1000$.
- Ví dụ



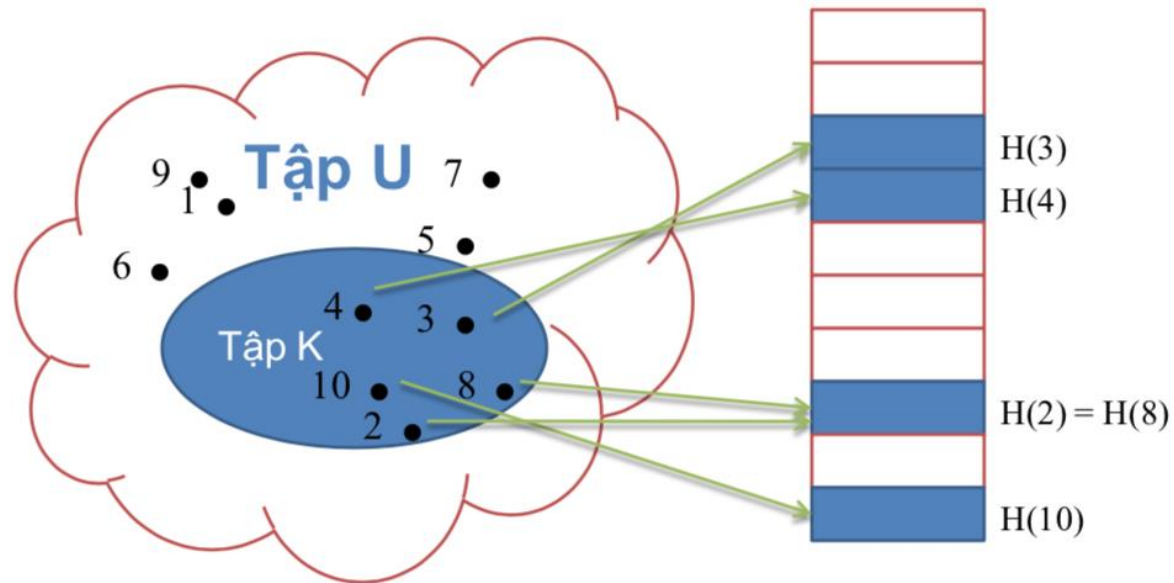
3.4. Tìm kiếm theo bảng băm

- **Khó khăn của hàm băm**

- Ví dụ : Sự đụng độ

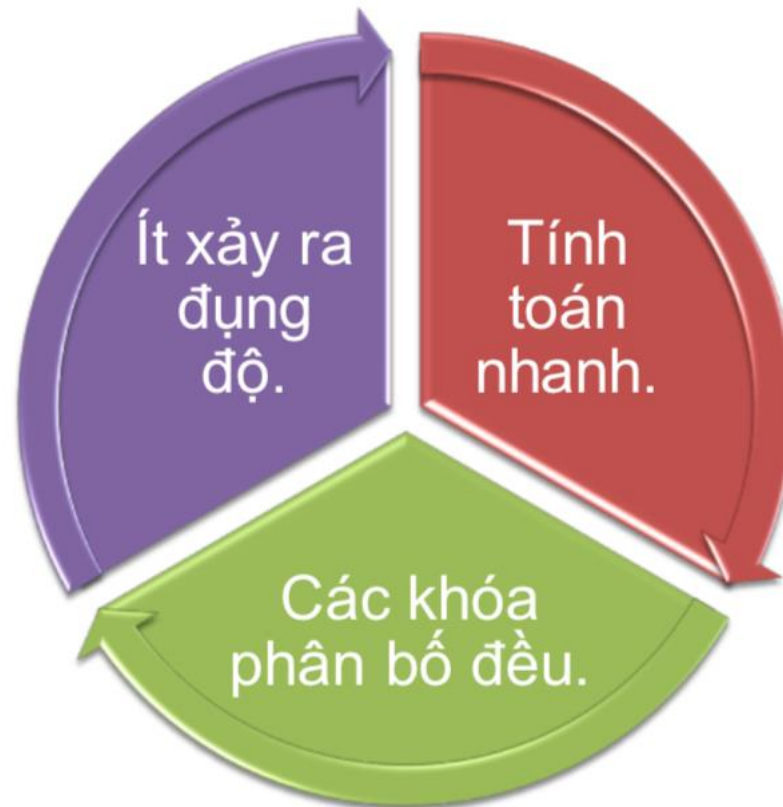
• $\exists k_1, k_2 \in K:$

$$k_1 \neq k_2, H(k_1) = H(k_2)$$



3.4. Tìm kiếm theo bảng băm

- Nhưng yêu cầu đối với hàm băm



3.4. Tìm kiếm theo bảng băm

- Xét lại ví dụ về danh sách sinh viên:
 - Với kích thước bảng là $M = 1000$, ta có thể chọn hàm băm như sau:
$$H(k) = k \bmod M.$$
 - Khóa này thỏa mãn yêu cầu tính toán nhanh và trải đều trên bảng.

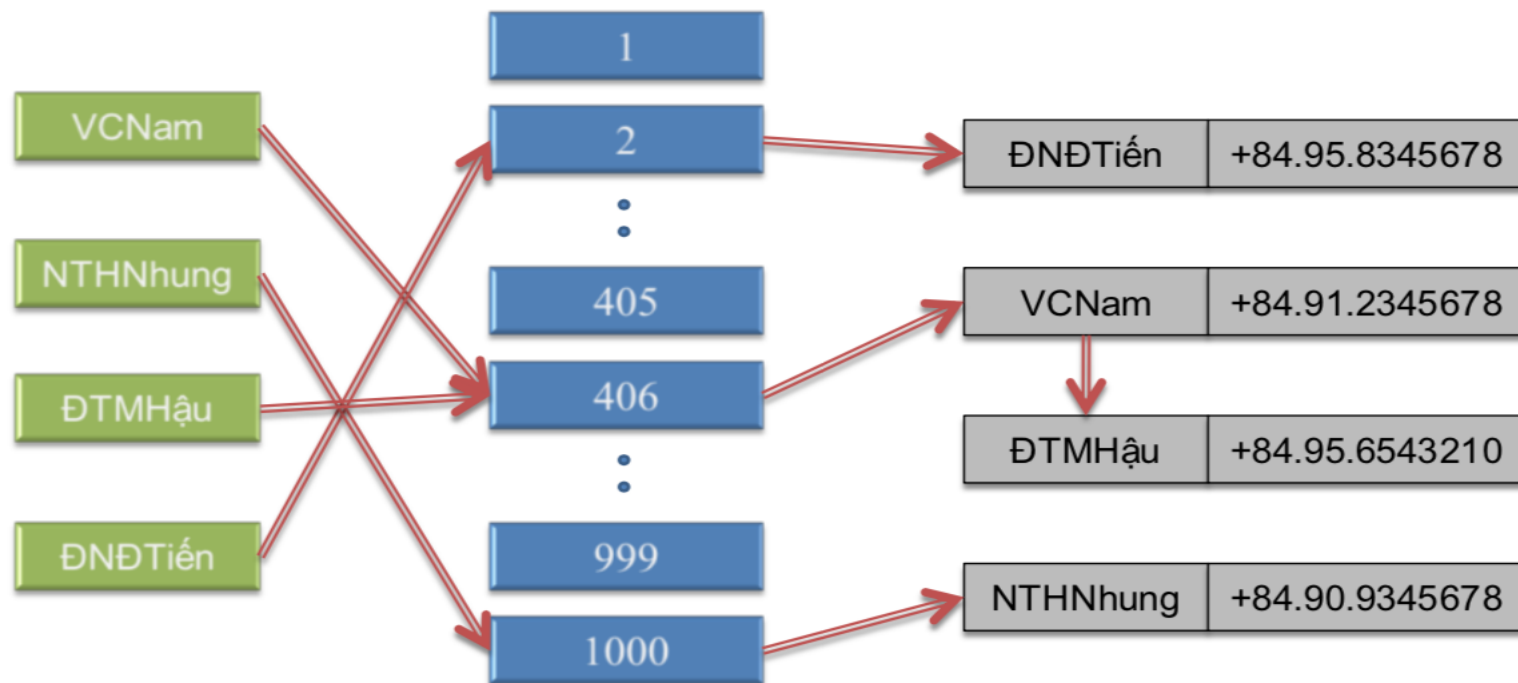
3.4. Tìm kiếm theo bảng băm

- Các phương pháp xử lý đụng độ
 - Phương pháp nối kết (dây chuyền) (Separate chaining)
 - Phương pháp địa chỉ mở (Open-addressing)

3.4. Tìm kiếm theo bảng băm

- Phương pháp nối kết

- Ứng với mỗi địa chỉ của bảng, ta có một danh sách liên kết chứa các phần tử có khóa khác nhau mà có cùng địa chỉ đó.
- Ta sẽ có danh sách (bảng băm) gồm M phần tử chứa địa chỉ đầu của các danh sách liên kết.

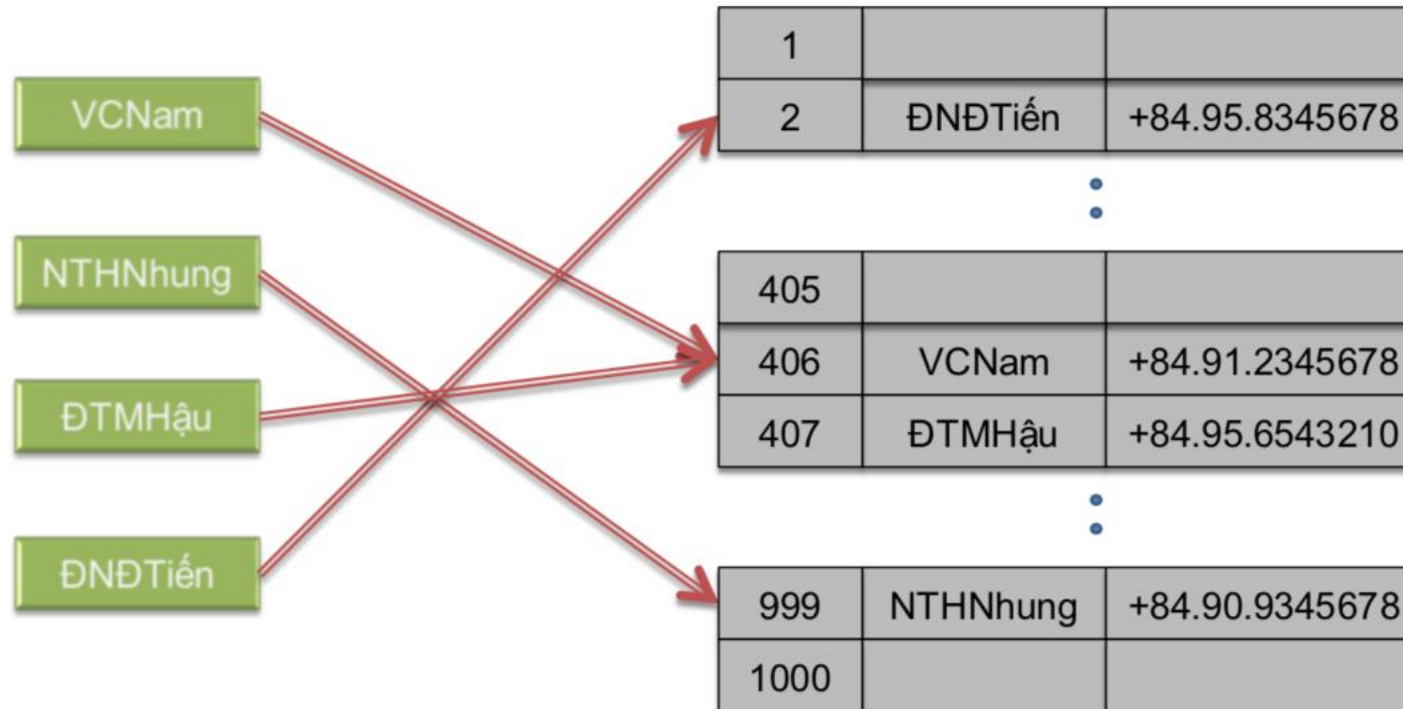


3.4. Tìm kiếm theo bảng băm

- Phương pháp địa chỉ mở
 - Tên gọi khác:
 - Phương pháp dò
 - Phương pháp thử
 - Ý tưởng
 - Khi đụng độ xảy ra, ta sẽ thử tìm đến vị trí kế tiếp nào đó trong bảng cho đến khi tìm thấy vị trí nào còn trống.
- Các cách thực hiện
 - Phương pháp dò tuyến tính (Linear probing)
 - Phương pháp dò bậc 2 (Quadratic probing)
 - Phương pháp băm kép (Double hashing)

3.4. Tìm kiếm theo bảng băm

- Phương pháp dò tuyến tính
 - Ý tưởng: $H(k, i) = (h(k) + i) \bmod M$



3.4. Tìm kiếm theo bảng băm

- Phương pháp dò bậc 2
 - $H(k, i) = (h(k) + i^2) \bmod M$
- Phương pháp băm kép:
 - $H(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod M$

3.4. Tìm kiếm theo bảng băm

- Ưu thế của phương pháp địa chỉ mở so với phương pháp nối kết
 - Đơn giản khi cài đặt.
 - Sử dụng cấu trúc dữ liệu cơ bản.
 - Phương pháp địa chỉ mở giải quyết được đụng độ nhưng lại có thể gây ra đụng độ mới.
 - Phương pháp nối kết không bị ảnh hưởng về tốc độ khi mảng gần đầy.
 - Ít tốn bộ nhớ khi mảng thưa (ít phần tử).

3.4. Tìm kiếm theo bảng băm

- Bài tập

1. Cho bảng băm có kích thước $M = 11$. Hàm băm: $h(k) = k \bmod M$. Dùng phương pháp địa chỉ mở. Cho biết kết quả sau khi thêm vào bảng băm các khóa 10, 22, 31, 4, 15, 28, 17, 88, 59, với 3 phương pháp xử lý đụng độ:

- Dò tuyến tính.
- Dò bậc 2.
- Băm kép $h_2(k) = (k \bmod 19) + 1$.

2. Cho từ điển Anh – Việt có 15.000 từ, hãy tổ chức cấu trúc dữ liệu bảng băm và cho biết hàm băm thích hợp giúp cho việc tra từ hiệu quả nhất.